

Counting Items in Warehouse Bins

UE24CS352A Machine Learning - Mini-Project Write-up
Pranav C (PES1UG24CS332) • Puneeth Yenneti (PES1UG24CS348)

1. Problem Statement

In a fulfilment centre, items are stored in bins and occasionally get misplaced, so the recorded inventory no longer matches what is physically in a bin. We set out to build a model that takes a colour photograph of a bin and predicts **how many items it contains (0 to 5)**, so that a mismatch with the recorded count can be flagged. We framed this as a 6-class image classification problem. Our project is inspired by the Stanford CS229 (2018) project “Amazon Inventory Reconciliation Using AI” (Bertorello et al.), whose best reported accuracy was 56.2% using 324K images, and we used that figure as a reference point for our own results.

2. Dataset

We used the public **Amazon Bin Image Dataset** (AWS Open Data, bucket *aft-vbi-pds*), which contains photos of bins from a real fulfilment centre. Each photo has a metadata file, and we used its *EXPECTED_QUANTITY* field as the label. We scanned image IDs 1-60,000 and kept the bins with at most 5 items, which gave us **48,206 images** (items 0-5: 1,454 / 5,483 / 10,205 / 11,948 / 10,672 / 8,444). The class counts showed us that the classes are imbalanced, so we kept two reference points in mind: random guessing gives 16.7% and always predicting the majority class gives 24.8%. We resized every image to 160×160, normalised it with ImageNet statistics, and made a stratified 70/20/10 train/validation/test split (33,743 / 9,642 / 4,821) with a fixed random seed. While looking through the photos we noticed that many are cluttered or covered by tape, and some are hard to count even for a person.

3. Approach

Classical baselines. We began with classical methods to see what deep learning would have to beat. We downsampled the images to 32×32×3, standardised them, reduced them with PCA to 100 components (90.7% of the variance), and trained logistic regression and an RBF-kernel SVM, choosing C on the validation set. **Deep learning.** We then moved to transfer learning with a ResNet18 pretrained on ImageNet, replacing its final layer with a 6-way classifier. We compared (A) a *frozen* backbone, where only the new layer is trained, with (B) *full fine-tuning* of all layers. Our first fine-tuning run overfit badly, so we added more data, random-shift and brightness augmentation on top of horizontal flips, AdamW with weight decay 0.01, label smoothing 0.1 and a cosine learning-rate schedule, and we kept the checkpoint with the best validation accuracy. At prediction time we average the original and horizontally flipped image (test-time augmentation).

4. Implementation Overview

We implemented everything in one Python notebook (PyTorch, torchvision, scikit-learn) and ran it on a free Colab T4 GPU. The notebook (i) downloads the images and labels over HTTPS in parallel, then resizes and caches them; (ii) creates the split and runs exploratory analysis; (iii) trains the PCA baselines; (iv) trains the ResNet18 models (fine-tuning: batch 64, learning rate 3e-4, 15 epochs, mixed precision); (v) evaluates on the held-out test set with accuracy, RMSE, per-class metrics, confusion matrices and misclassified examples; and (vi) saves the trained weights. We also wrote a command-line *demo.py* that predicts the item count for any bin photo on a CPU. The code, README, weights and sample images are in our private GitHub repository.

5. Results

Model	Val acc.	Test acc.	Test RMSE
Random guess / majority class	-	16.7% / 24.8%	-
Logistic regression (PCA features)	28.8%	27.2%	1.433
SVM, RBF kernel (PCA features)	29.4%	29.5%	1.340
ResNet18, frozen backbone	32.9%	32.4%	1.318
ResNet18, fully fine-tuned (final model)	50.4%	48.6%	1.012

Table 1: Held-out test results (4,821 images). Our first run (21K images, 128 px, plain Adam, flip only) reached 42.0%; the changes described above raised it to 48.6%.

6. Conclusions

We found that fine-tuned transfer learning clearly beats classical methods on raw-pixel features: 48.6% against 29.5% for our best SVM, and about 3× random guessing. Fine-tuning also beat the frozen backbone, which we think is because ImageNet features do not fully fit cluttered, taped bins. Our test accuracy fell as the true count rose (96.6%, 82.1%, 60.3%, 41.1%, 33.8% and 33.7% for 0 to 5 items): empty and near-empty bins are easy, while occlusion, tape and similar-looking items make larger counts hard. The model still overfits, with about 83% training accuracy against about 50% validation at the end of training, so we believe more data and regularisation were the key levers, although we changed several things at once and did not isolate each one. We fall short of the paper's 56.2%, which we attribute mainly to using about 48K rather than 324K images, and smaller images. **Future work:** with more time we would train on the full dataset, clean unusable images, use class weighting, try larger models or ensembles, and test counting-specific or detection-based methods.

References: Bertorello, Sripada, Dendumrongsup, *Amazon Inventory Reconciliation Using AI*, Stanford CS229, 2018; Amazon Bin Image Dataset, AWS Open Data; He et al., *Deep Residual Learning for Image Recognition*, 2015; Loshchilov & Hutter, *Decoupled Weight Decay Regularization*, 2017.